COMPLAINT COMPASS · JUDGE Q&A PREP

“How are you going to scale it?”

What to say, why it is true, and the numbers behind it. Tags: MEASURED on the live run of 1,500 complaints; PROJECTED from measured timings, not yet built or load-tested; ASSUMPTION our own input.

1. The answer in 30 seconds

> “The slow part of our pipeline is waiting on the network for a model to answer, and that part has no shared state, so we scale it by running more copies. The only shared state is the list of known issues, and it is already split by business and by area, like payments or delivery. Each of those gets one dedicated worker, and they all run side by side. Only about 4% of complaints reach an LLM, so cost grows in a straight line: about $433 a month in model calls at 10 million complaints.”

Then stop. Let the judge choose what to dig into, and use the sections below.

2. What “scaling” has to cover

Judges usually mean one of five things. Know which one you are answering.

Dimension	The question behind it	Our short answer
Volume	Can it handle millions a month?	Add intake workers; each area runs in parallel (section 4).
Spikes	What happens on an outage day?	A queue absorbs it; most spike complaints match for free (section 6).
More businesses	Can one system serve many companies?	Each business is its own settings file and its own partitions.
Cost	Does cost explode with volume?	No: linear, $4.33 per 100,000 in model calls (section 7).
Reliability	What if a model provider fails?	Only ~4% need the LLM; Jev needs a fallback (section 6).

3. Why we can’t just “make the prototype bigger”

Today the prototype is one program with one database file (SQLite). It holds the list of known issues in memory and lets only one complaint update it at a time, using a lock (`api.py`, line 38).

> **Why that lock exists: the key idea to explain.** The issue list changes while complaints are being processed. Suppose three customers report a brand-new problem at the same moment. If three workers handled them at once, each would see “no matching issue” and each might create its own new issue: three duplicates of one problem. So whoever adds to the issue list must work **one complaint at a time, in order**. Anything that reads or writes the issue list needs order; anything that does not, does not.

So the question is not “how do we make it bigger” but “which parts need that order, and which don’t?”

4. How it scales: split the work by what needs shared state

Where the time actually goes MEASURED

Step	Time	Needs the issue list?
Jev gate (every complaint)	0.53 s	No
Local similarity match	~0.02 s	Yes
Jev verify (35% of complaints)	0.54 s	Yes
LLM decision (~4%)	1.90 s	Yes

Source: HANDOFF.md §4, from per-call timings in the live run.

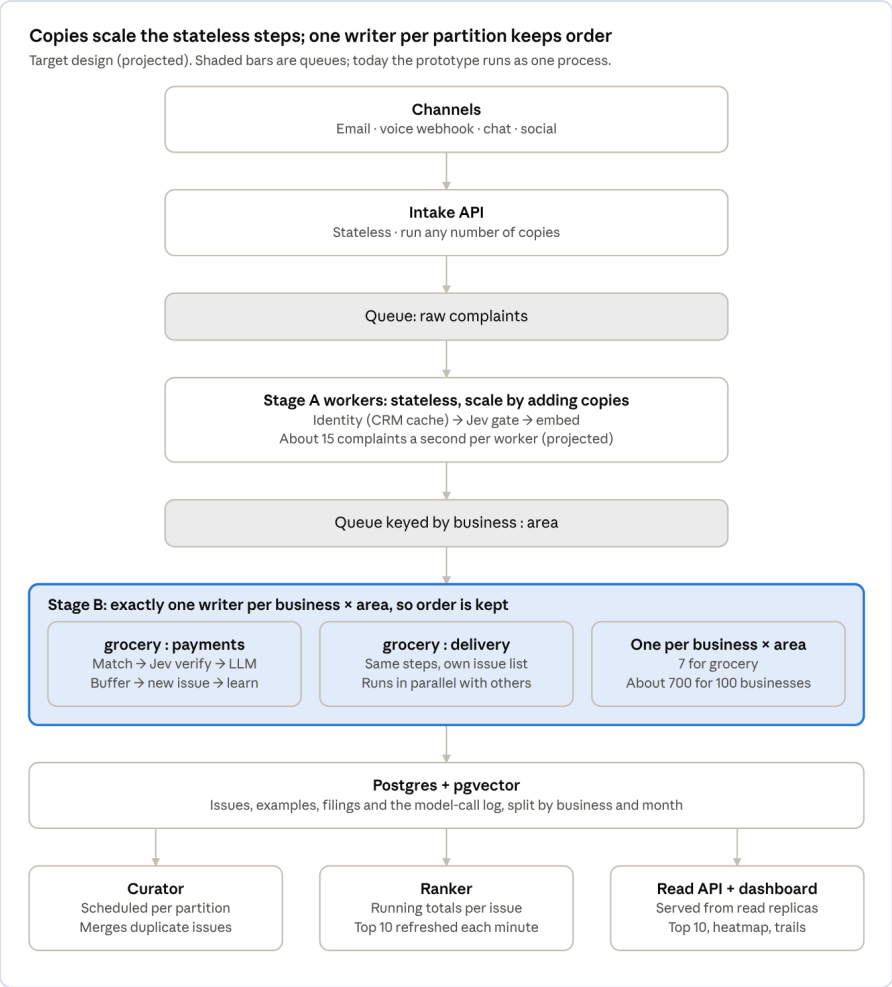
The insight

Almost all of a typical complaint's 0.55 s is the gate, which is waiting on the network, and the gate only looks at the complaint itself. That means the **slowest step is also the easiest to scale**: run as many copies as needed.

The steps that need the issue list are short for most complaints. They must run in order, but only **within one business area**, because a complaint is only compared with issues in its own area (`engine.py`, `_components`).

So the pipeline splits into two stages, joined by a queue

- **Stage A: no shared state, scale by copies.** Identify the customer, run the Jev gate, turn the text into numbers. Today's engine already runs the gate 8 at a time (`prefetch_gate`). In production we run many such workers.
- **Stage B: one writer per business × area.** Match, verify, LLM, and creating new issues. Each partition (for example "grocery : payments") has exactly one worker, so its issue list stays in order. Different partitions run side by side. Grocery has 7 areas, so 7 partitions; 100 businesses would have about 700.
- **Storage:** Postgres with a vector index (pgvector) replaces the single SQLite file. The ranker keeps running totals per issue instead of recounting everything.



Target architecture **PROJECTED**. Point at this and walk it top to bottom.

A complaint that fits two areas

If the gate is unsure between two areas (say payments and refunds, within 0.15 of each other), the payments worker also reads the refunds issue list from a recent snapshot, but it only writes to payments. Reading never needs the lock.

---

5. Follow one complaint through the scaled system

1. A customer emails: “Charged twice for one order after the app update.” The intake API accepts it and puts it on the first queue.
2. Any free Stage A worker picks it up: links it to the customer’s record, runs the Jev gate (complaint: yes; area: payments; money lost: yes), turns the text into numbers.
3. It goes onto the second queue under the key `grocery : payments`.
4. The single payments worker takes it in turn and compares it with the known payments issues. A close match? Filed, free. Unsure? A Jev yes/no check, then the LLM (Claude Haiku 5.5) only if still unsure. No match? It waits in the payments tray until 3 different customers report the same thing, then becomes a new issue.
5. The filing is written to Postgres; the issue’s running totals update; the Top 10 refreshes within a minute.

6. The numbers, and how each one is worked out

Claim	How it is worked out	Type
One Stage A worker handles ~15 complaints a second (~1.3M a day)	8 gate calls at a time ÷ 0.53 s each ≈ 15 a second; × 86,400 seconds	PROJECTED
One partition handles ~3.5 a second (~300,000 a day)	Average Stage B time = 0.02 s + 35% × 0.54 s + 3.9% × 1.90 s ≈ 0.28 s, one at a time	PROJECTED
Grocery's 7 partitions handle ~24 a second together	7 × 3.5, if load were even across areas (it won't be exactly)	PROJECTED
All Indian quick commerce averages ~1.8 complaints a second	7.8M orders a day (RedSeer) × 2% complaint rate ÷ 86,400 s	ASSUMPTION on the 2%
LLM calls at 10M a month: ~0.15 a second	10M × 3.9% ÷ 2.6M seconds in a month	PROJECTED

> The line to remember: **at our 2% assumption, the entire Indian quick-commerce market averages under 2 complaints a second; one intake worker handles about 15, and one business's 7 partitions about 24.** Scaling the beachhead is a handful of workers, not a rebuild.

What happens when things go wrong

Scenario	What happens
A spike: a payment outage brings 10× complaints	The queue absorbs the burst. Most spike complaints describe the same fault, so after the first few they match for free at the 0.02 s step. If the payments partition still falls behind, split it (for example by region).
The LLM provider is slow or down	Only ~4% of complaints wait; the other 96% are still filed. Waiting ones queue and resume.
Jev is down	Today this stops intake: Jev is on the critical path. The fix is a degraded mode: route by similarity alone, label those filings, and re-check them when Jev returns.
A new business signs up	Write its settings file (areas, known issues, questions). Its partitions start; nobody else is affected. Jev takes labels per request, so no retraining.

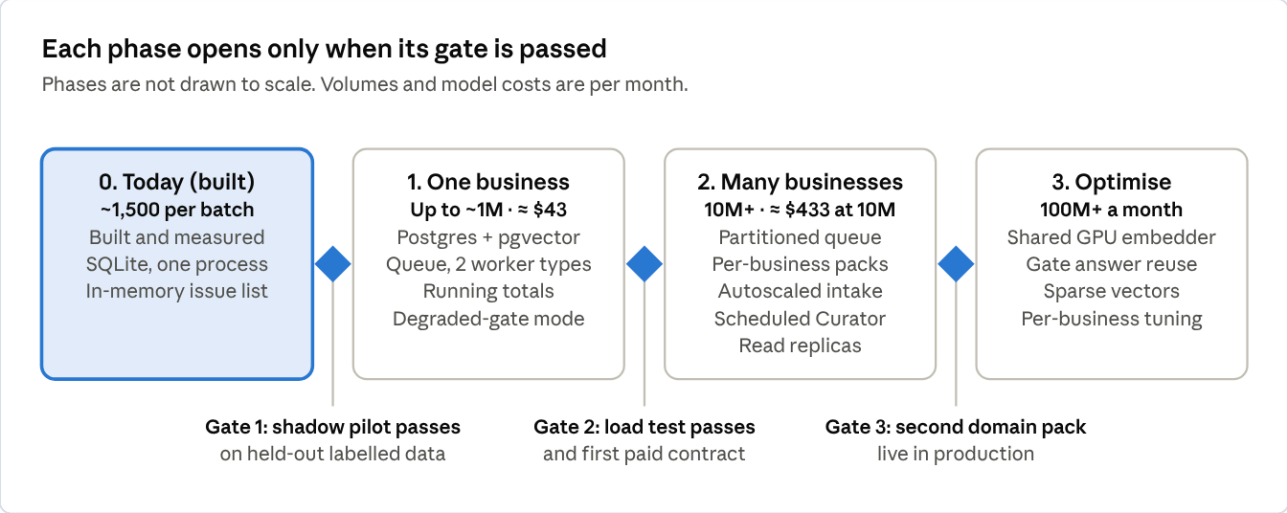
7. Cost at scale, with the exact models

Our funnel uses **Jev 1.14** (a decision model, not an LLM) for the gate and yes/no checks, and **Claude Haiku 5.5** for the ~4% of hard cases, naming new issues and merging duplicates, all via OpenRouter at list prices. The comparison is an "LLM reads every complaint" design, priced (not run) for **Claude Haiku 5.5** and **Claude Sonnet 5.5**.

Complaints a month	Our model cost rate MEASURED	Claude Haiku 5.5, best caching	Claude Sonnet 5.5, no caching
1M	$43	$123	$5,940
10M	$433	$1,232	$59,400
100M	$4,330	$12,320	$594,000

Cost per 100,000 measured on the live run × volume. Excludes hosting, which is not yet measured. List prices per million tokens: Haiku 5.5 $0.10 in / $0.50 out; Sonnet 5.5 $2 / $10. $4.33 is our cost, not our price.

8. When we build it



Each phase starts only when its gate is passed. "We don't build capacity ahead of evidence."

9. Say these limits before a judge does

- **None of the scaled design is built yet,** and nothing has been load-tested. Every throughput figure is worked out from measured timings.
- **Model-provider rate limits at volume are unknown.** That is why a load test is the gate before the first contract.
- **Scale is not our biggest risk; accuracy on real, messy complaints is.** Our test data is synthetic. That is why the first step is a shadow pilot.

10. Quick answers to follow-ups

They ask	You say
"Why not just run many copies of the whole thing?"	"Two copies could each create the same new issue, and counts would double. So we copy only the parts that don't touch the issue list."
"What breaks first?"	"The single writer and the SQLite file. The plan replaces both: one writer per area, and Postgres."
"Why split by area?"	"A complaint is only ever compared with issues in its own area, so areas never need to wait for each other."
"One hot area, like payments?"	"One partition handles about 3.5 a second, roughly 300,000 a day. If that's not enough, we split the area further, for example by region."
"Is $4.33 what you charge?"	"No, it's what we pay Jev and Claude per 100,000 complaints. Price is set on value; we propose a platform fee plus a charge per complaint."
"What if real data is messier?"	"Cost rises in a straight line, slowly. If 20% needed the LLM, cost goes from $4.33 to about $6.40 per 100,000."
"Have you load-tested it?"	"No. The figures come from measured timings. A load test is our gate before any contract."

11. Three rules, and how to close

1. **Say “measured” or “projected” every time.** Separating them yourself earns trust.
2. **Explain the lock first** (section 3). Once a judge sees why order matters, the rest of the design is obvious.
3. **Point at the architecture slide;** don’t recite it.

> “So: the slow part scales by adding copies, the part that needs order is already split by area, the costs are measured, and we only build each phase once the last one has proven itself.”

Sources: backend/runs/grocery_report.json; HANDOFF.md §4; SCALABILITY.md; backend/app/api.py and pipeline/engine.py; the Market and Scale report (RedSeer via GAEE for order volume).